

CSCE 669 COMPUTATIONAL OPTIMIZATION · MODULE 06 · LECTURE NOTES

Constrained Optimisation

Optimality conditions when you cannot go everywhere.

MODULE OUTCOMES

- State and apply the KKT conditions.
- Explain the geometric meaning of the stationarity condition.
- Apply projected and penalty methods.
- Explain augmented Lagrangian and ADMM.
- Recognise when constraints can be eliminated.

1 The KKT conditions

At an optimum x^* with multipliers λ^* , ν^* :

1. STATIONARITY

$$\text{grad } f(x^*) + \sum \lambda_i \text{grad } g_i(x^*) + \sum \nu_j \text{grad } h_j(x^*) = 0$$

the objective gradient is a combination of the ACTIVE constraint gradients

2. PRIMAL FEASIBILITY $g_i(x^*) \leq 0, \quad h_j(x^*) = 0$

3. DUAL FEASIBILITY $\lambda_i \geq 0$

inequality prices cannot be negative -- you would not pay to be MORE constrained

4. COMPLEMENTARY SLACKNESS $\lambda_i * g_i(x^*) = 0$ (Module 04 section 2)

CONVEX + constraint qualification: KKT is NECESSARY AND SUFFICIENT. Finding a KKT point SOLVES the problem.

NON-CONVEX: KKT is only NECESSARY. A KKT point may be a saddle or a local maximum.

The convex case is the useful one. KKT converts an optimisation problem into a system of equations and inequalities — which can sometimes be solved directly by hand for small problems, and which is what interior point methods (Module 07) solve numerically. **The necessary-versus-sufficient distinction is the thing to retain:** in the non-convex case a solver reporting 'KKT conditions satisfied' has found a stationary point and established nothing about optimality.

STATIONARITY SAYS THE GRADIENTS BALANCE

At an unconstrained optimum the gradient is zero — there is no direction in which the objective improves.

At a constrained optimum the gradient need not be zero. It must instead point 'into' the constraints, so that every direction that would improve the objective is infeasible — blocked by a constraint.

Formally: the objective gradient lies in the cone spanned by the gradients of the *active* constraints. That is exactly what the stationarity equation asserts, with the multipliers as the non-negative coefficients.

Picture a ball rolling downhill into a corner. It comes to rest not because the slope vanished but because the walls push back exactly enough to cancel gravity — **and λ measures how hard each wall is pushing**, which is Module 04 §2's shadow price arriving as a physical quantity. **A wall the ball is not touching pushes with force zero**, which is complementary slackness.

2 Methods

Method	Idea	Character
Elimination	Use the equality constraints to substitute variables away.	The best option when it is available — it reduces the dimension and removes the constraint entirely rather than handling it.
Projection	Take an unconstrained step, then project back onto the feasible set.	Requires a cheap projection , so it is limited to simple sets — boxes, balls, the simplex, the non-negative orthant. Where it applies it is excellent.
Penalty	Add ρ times the squared violation to the objective and minimise unconstrained.	Becomes ill-conditioned as ρ grows — see the callout.
Barrier	Add a term that goes to $+\infty$ at the boundary, so iterates stay strictly interior.	The basis of interior point methods (Module 07).
Augmented Lagrangian	A penalty term plus explicit multiplier estimates, updated each outer iteration.	Achieves exact convergence at a finite, moderate penalty , avoiding the conditioning blow-up.
ADMM	Split the variables so each subproblem is easy, and alternate between them.	Decomposes across data and across terms; excellent for large structured problems (§3).

WHY THE PURE PENALTY METHOD FAILS

Add $\rho \cdot (\text{violation})^2$ to the objective and minimise it as an unconstrained problem. As $\rho \rightarrow \infty$ the solution approaches the constrained optimum, which is the right limiting behaviour.

But the Hessian's condition number grows with ρ . The penalty adds enormous curvature in the constraint directions and none elsewhere, so κ **grows without bound and each unconstrained subproblem becomes progressively harder to solve** (Module 05 §1) — the iteration count blows up exactly as the answer becomes accurate.

And at any finite ρ the answer is never exactly feasible. You are trading accuracy against conditioning with no setting that gives both.

The augmented Lagrangian resolves this by adding explicit multiplier estimates that are updated after each inner solve: the multipliers absorb what the penalty would otherwise have to force, **so exact convergence is achieved at a finite and moderate ρ . Which is duality doing practical work** — the multipliers of Module 04 turning out to be the fix for a numerical problem.

3 ADMM

```

Rewrite  min f(x) + g(z)  subject to  Ax + Bz = c
-- SPLIT the objective so that each part is easy ALONE.

Then alternate:
  x <- argmin_x L_rho(x, z, u)    # f's part only
  z <- argmin_z L_rho(x, z, u)    # g's part only
  u <- u + (Ax + Bz - c)          # dual update

WHY IT IS USEFUL
* each subproblem touches only ONE of f and g, so each can
  use whatever method suits -- often a closed form
* it DECOMPOSES: if f splits across data, the x-update is
  embarrassingly parallel (CSCE 735)
* convergence is modest -- moderate accuracy fast, high
  accuracy slowly, which suits machine learning well

LASSO IN ADMM
  x-update:  ridge regression (closed form)
  z-update:  SOFT THRESHOLDING (Module 05 section 3)
  trivial steps; the coupling is carried by u

```

The pattern is: split the problem so each piece is easy, and let the multipliers handle the coupling. Boyd's 2011 monograph collected the method and its applications and made it the standard tool for large-scale structured problems — **and it remains underused outside the optimisation community** relative to how well it fits distributed machine learning, where the data split is natural and moderate accuracy is sufficient.

4 Recognising removable constraints

- **Equality constraints can frequently be substituted away**, reducing the problem's dimension and eliminating the constraint entirely. **Always worth attempting when the substitution is clean** — it makes every subsequent step easier.
- **Simple bounds become projections**, which for a box or the non-negative orthant cost one clamp per coordinate — effectively free, and projected gradient descent then applies directly.
- **A simplex constraint (non-negative, summing to one) becomes a softmax reparameterisation.** Optimise unconstrained over logits and map through the softmax; the constraint is satisfied by construction. **This is why neural network classifiers optimise freely (CSCE 636) — the probability simplex constraint was absorbed into the parameterisation and nobody had to mention it**, which is worth noticing as an instance of a general technique.
- **Orthogonality constraints define a manifold** (the Stiefel manifold), which has its own gradient methods — Riemannian optimisation — that respect the geometry rather than projecting onto it.

- **And a redundant constraint can simply be deleted.** Presolve (Module 03 §4) does this automatically, and **modellers add redundant constraints routinely by accident** — expressing the same restriction twice in different forms, which costs nothing in correctness and a great deal in solve time.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Boyd & Vandenberghe — Convex Optimization, chapters 5 and 10 (free)	KKT and the constrained methods of §1 and §2, with the geometric interpretation.
Boyd et al. — Distributed Optimization and Statistical Learning via ADMM (free)	The §3 monograph. Comprehensive, readable, and full of worked formulations including the lasso.
Nocedal & Wright — Numerical Optimization, chapters 12 and 17	The augmented Lagrangian development of §2, with the conditioning analysis.
Absil, Mahony & Sepulchre — Optimization Algorithms on Matrix Manifolds (free PDF)	The manifold case of §4, free from the publisher.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write the KKT conditions** for your project model and check them at the solver's solution.
- 2 **Solve a small constrained problem by hand from KKT** and verify against a solver.
- 3 **Find a KKT point of a non-convex problem that is not optimal.**
- 4 Implement projected gradient descent for a box constraint.
- 5 **Implement the pure penalty method** and plot the condition number and iteration count against ρ .
- 6 **Implement the augmented Lagrangian** and show convergence at finite ρ .
- 7 **Implement ADMM for the lasso** and verify the z-update is soft thresholding.
- 8 Compare ADMM against proximal gradient on the same lasso instance.
- 9 **Eliminate an equality constraint by substitution** and compare solve times.
- 10 **Reparameterise a simplex constraint as a softmax** and optimise unconstrained.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State the four KKT conditions and what each says.
- 2 When is KKT sufficient, and when only necessary?
- 3 Explain stationarity geometrically.
- 4 Compare six methods for handling constraints.
- 5 Why does the pure penalty method fail, and what fixes it?
- 6 Describe ADMM and say why it decomposes.
- 7 Give the ADMM steps for the lasso.
- 8 Name five constraints that can be eliminated or reparameterised.